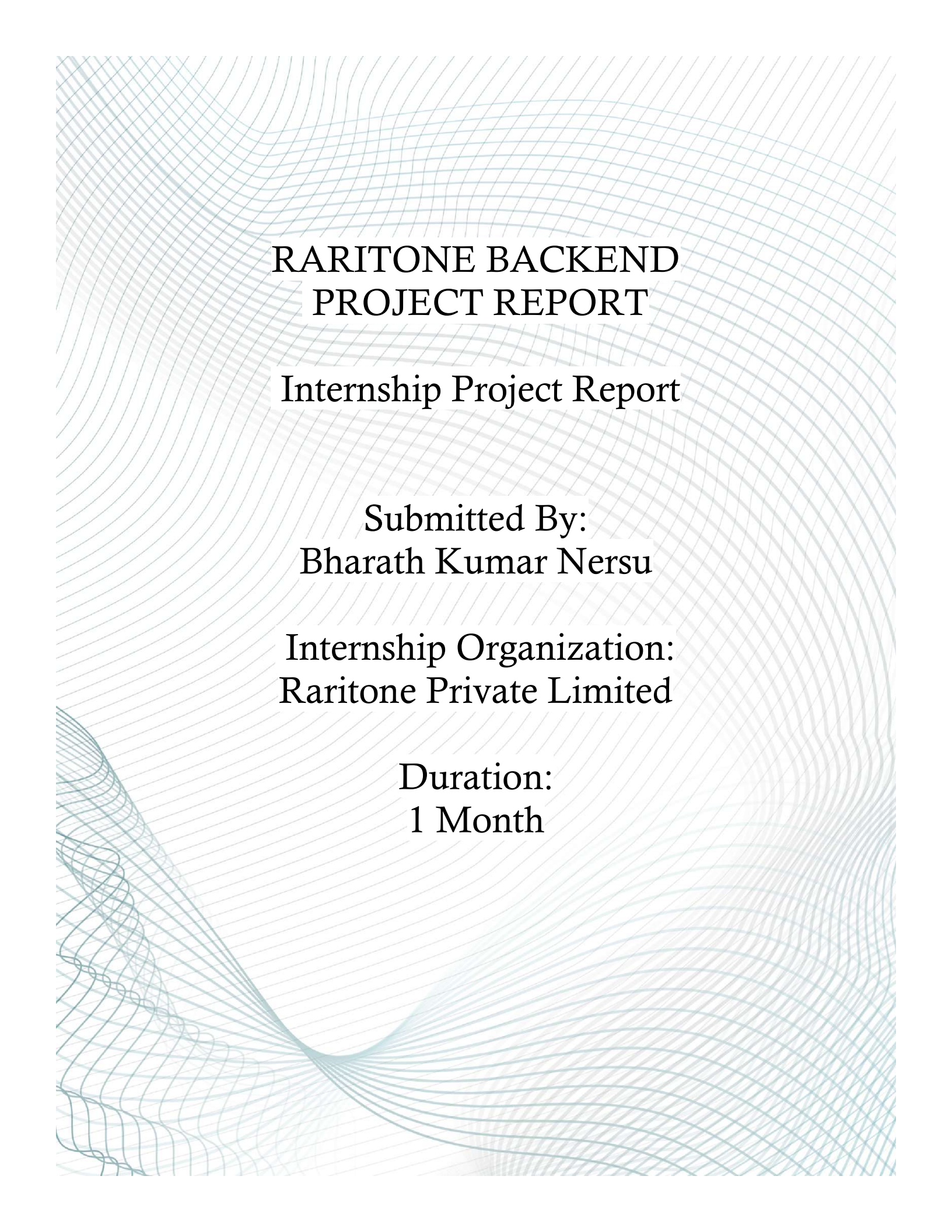




RARITONE BACKEND PROJECT REPORT

Internship Project Report

Submitted By:
Bharath Kumar Nersu

Internship Organization:
Raritone Private Limited

Duration:
1 Month

DECLARATION

I hereby declare that the project report entitled "Raritone Backend Project" submitted in partial fulfillment of the internship requirements is my original work. The information and results presented in this report are based on the work carried out during the internship period.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the management and technical team of Raritone for providing me with the opportunity to work on this project. Their guidance, support, and valuable insights helped me gain practical knowledge in backend development, database management, API design, and software architecture.

I also thank my faculty members and mentors for their encouragement and support throughout the internship period.

ABSTRACT

Raritone is an innovative virtual fashion platform developed to enhance the online shopping experience through personalization, intelligent recommendations, and virtual fashion technologies. Traditional e-commerce platforms face challenges such as inaccurate size selection, limited personalization, and the inability for customers to visualize how products will appear before purchase. These limitations frequently result in customer dissatisfaction and increased product return rates.

The primary objective of this project was to design and develop a secure, scalable backend system capable of supporting various functionalities required for a virtual fashion ecosystem. The backend was built using **Node.js** and **Express.js**, with **MongoDB** as the primary database. The architecture follows a modular design approach to ensure maintainability, flexibility, and future scalability.

The system includes modules such as user authentication, profile management, product management, wishlist, wardrobe management, shopping cart, order processing, measurement storage, avatar management, and virtual try-on support. Security was implemented using JWT authentication, role-based access control, input validation, rate limiting, and secure middleware configurations. Media handling was integrated through **Multer**, **Sharp**, and **Cloudinary** for efficient image storage and delivery.

In addition to current functionalities, the backend architecture was designed with future AI integration in mind – AI-powered avatar generation, personalized recommendations, and virtual try-on processing can be added with minimal changes.

Overall, the Raritone Backend Project successfully establishes a robust foundation for a next-generation virtual fashion platform.

1. INTRODUCTION

The fashion e-commerce industry faces challenges such as inaccurate size selection, lack of personalization, and uncertainty regarding how products will look on customers. These issues often result in poor customer satisfaction and increased product returns.

Raritone addresses these problems by providing a smart virtual fashion platform capable of supporting virtual try-on technology, personalized recommendations, avatar generation, and user-specific measurements.

The backend acts as the foundation of the platform, managing user data, product information, security, business logic, and future AI integrations.

2. PROBLEM STATEMENT

Traditional online fashion shopping platforms face several limitations:

- Customers cannot visualize outfits before purchasing.
- Size and fit recommendations are often inaccurate.
- Personalized recommendations are limited.
- Managing wardrobe and fashion preferences is difficult.
- Product return rates are high due to fitting issues.

The project aims to solve these challenges by creating a backend architecture capable of supporting AI-powered virtual fashion experiences.

3. OBJECTIVES

The primary objectives of the project are:

- Develop a secure backend API system.
- Implement authentication and authorization (JWT + Google OAuth).
- Manage products, user profiles, and orders.
- Store user body measurements for size recommendations.
- Support virtual try-on workflows and real-time updates ([Socket.IO](#)).
- Manage avatars and personalization data.
- Create scalable architecture for AI integration.
- Ensure data security with rate limiting, Helmet, and validation.
- Build a modular backend structure suitable for future expansion.

4. TECHNOLOGY STACK

Backend Framework

- **Node.js** – JavaScript runtime
- **Express.js** – Web framework for REST APIs

Database

- **MongoDB** – NoSQL document database
- **Mongoose** – ODM for schema modeling

Authentication & Security

- **JWT (jsonwebtoken)** – Access & refresh tokens
- **bcryptjs** – Password hashing
- **Google OAuth (google-auth-library)** – Social login
- **Helmet** – Secure HTTP headers
- **express-rate-limit** – Rate limiting (100 requests per 15 minutes)
- **CORS** – Cross-origin resource sharing

Media Processing

- **Multer** – File upload handling (memory storage)
- **Sharp** – Image compression and resizing
- **Cloudinary** – Cloud storage + CDN

Utilities

- **dotenv** – Environment variables
- **Joi** – Request validation
- **Nodemailer** – Password reset emails
- **Morgan** – HTTP request logging
- **compression** – Gzip response compression
- [Socket.IO](#) – Real-time communication

Development Tools

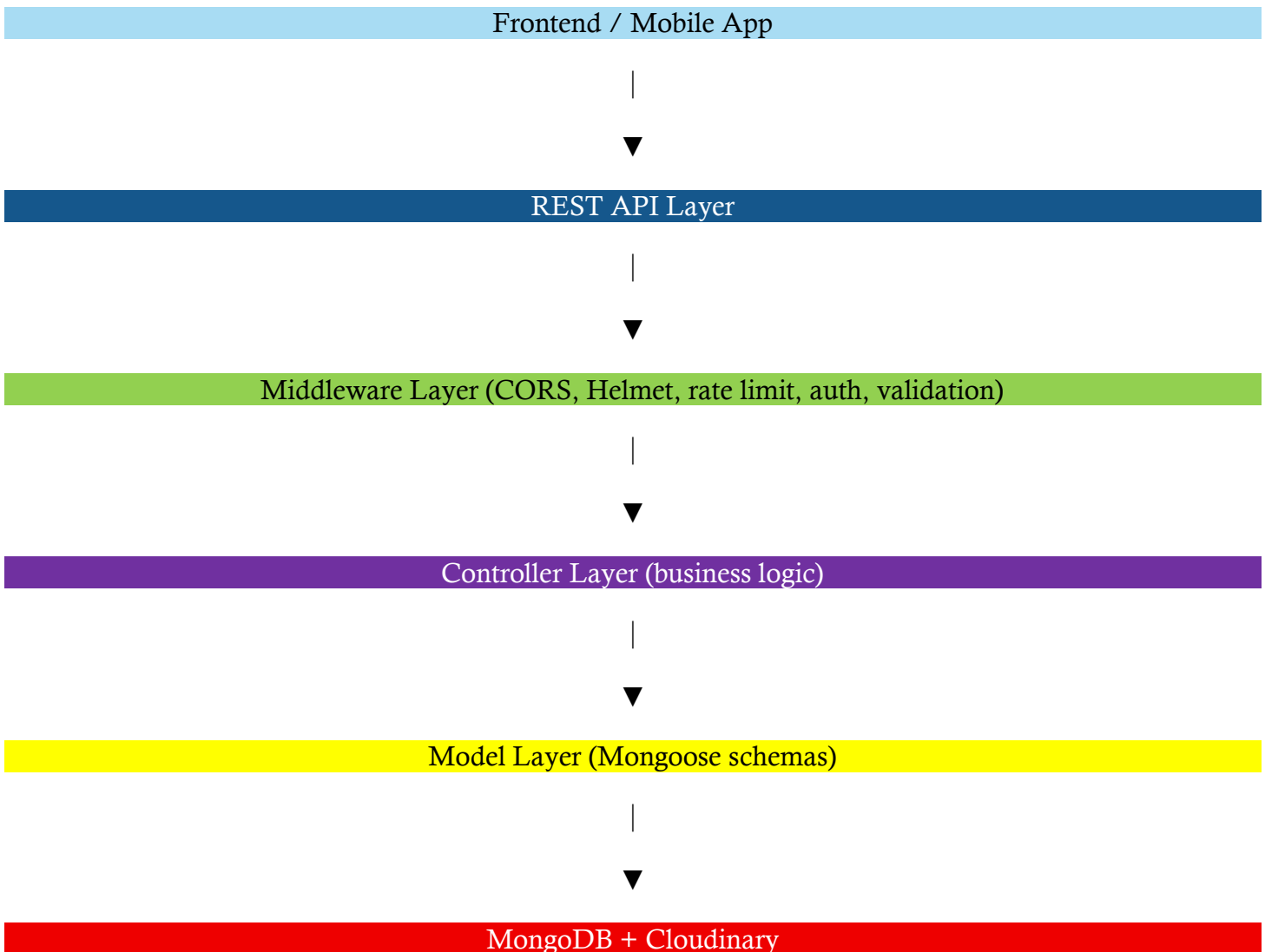
- **nodemon** – Hot reload during development
- **Git & GitHub** – Version control

Future Technologies Researched

- Redis (caching)
- BullMQ / RabbitMQ (queues)
- Docker & Kubernetes (containerization)
- AWS (deployment)

5. SYSTEM ARCHITECTURE

The backend follows a **modular layered architecture**:



Advantages of this architecture:

- Easy maintenance
- Better scalability
- Modular development

- Improved security
- AI integration readiness

6. DATABASE DESIGN

MongoDB was used as the primary database. The following collections were implemented:

Collection	Purpose
User	Authentication, roles, password reset tokens
Profile	User preferences (theme, language, notifications), bio
Product	Product catalog (title, price, category, images, stock)
Wishlist	User-saved products
Wardrobe	User’s digital clothing collection
Cart	Shopping cart (one per user)
Order	Completed purchases with shipping address and status
Measurement	Body measurements (chest, waist, shoulder, hip, height)
TryOn	Virtual try-on records (original + generated images)
Image	Generic image storage (profile, avatar, product, try-on)
ProductMapping	AI recommendation metadata (supported body types, compatibility)

Relationships (using Mongoose ref and populate):

- User → Profile (one-to-one)
- User → Wishlist, Wardrobe, Cart, Orders, Measurements, TryOn (one-to-many)
- Wishlist/Cart/Order → Product (many-to-one)
- ProductMapping → Product (many-to-one)

Indexes were added on email, userId, category, and createdAt for query performance.

7. MODULES DEVELOPED

Authentication Module

- User registration, login, logout
- JWT access token (15 min) + refresh token (7 days)
- Google OAuth login
- Forgot password / reset password (Nodemailer)
- Role-based access (USER, ADMIN, SUPER_ADMIN)

Product Module

- CRUD operations (admin only for write)
- Image upload with Sharp compression + Cloudinary
- Search and filter (keyword, category, price range)
- Automatic cleanup of old images on update/delete

Cart Module

- Add product (creates cart if not exists)
- Update quantity (increments if product already in cart)
- Remove product
- Get cart with populated product details

Order Module

- Create order from cart data
- Order status workflow: PENDING → PLACED → SHIPPED → DELIVERED / CANCELLED
- Get all user orders, get single order
- Update order status, delete order

Profile Module

- Get/update profile (bio)
- Update preferences (theme, language, notifications)
- Upload/delete profile image and avatar image

Wardrobe Module

- Add clothing item (name, category, color, brand, image)
- Get all wardrobe items
- Delete item (with Cloudinary cleanup)

Wishlist Module

- Add product to wishlist (prevents duplicates)
- Remove product from wishlist
- Get wishlist with populated product details

Measurement Module

- Save/update body measurements (chest, waist, shoulder, hip, height)
- Get measurements
- Delete measurements

Try-On Module

- Upload two images (original user photo + generated garment image)
- Store in Cloudinary, create TryOn record
- Get try-on history
- Delete try-on (with image cleanup)
- Mock AI endpoints for future integration

Image Module

- Upload general images (profile, avatar, body, product, try-on)
- Get, update, delete images
- Automatic Cloudinary deletion on replace/delete

Avatar Module (partial – routes defined, controller planned)

- Create, get, delete avatars (future AI avatar generation)

8. SECURITY IMPLEMENTATION

The backend includes multiple security layers:

JWT Authentication

- Access token expires in 15 minutes, refresh token in 7 days.
- authMiddleware verifies token and attaches user to req.user.

Role-Based Access Control (RBAC)

- **USER** – access own resources.
- **ADMIN** – full product management.
- **SUPER_ADMIN** – user management + everything else.

Helmet

Sets secure HTTP headers (CSP, X-XSS-Protection, X-Frame-Options, etc.)

Rate Limiting

- 100 requests per IP per 15 minutes (global).
- Prevents brute-force and DDoS attacks.

Input Validation (Joi)

- Validates request bodies before reaching controllers.
- Returns 400 with detailed error messages.

Error Handling Middleware

- Centralized error handler catches CastError, duplicate keys, validation errors, JWT errors.
- Returns consistent JSON error responses.

Password Security

- Bcrypt with 10 salt rounds.
- Reset tokens hashed with SHA256, expire after 10 minutes.

9. MEDIA PROCESSING WORKFLOW

The image upload pipeline is as follows:

1. **Multer** receives file (memory storage, 5MB limit, only JPEG/PNG/WEBP).
2. **Sharp** compresses and resizes the image (800px width, JPEG quality 80%).
3. **Cloudinary** uploads the buffer (no temporary files).
4. **URL optimization** adds transformations: w_500,h_500,c_fill,f_auto,q_auto for CDN delivery.
5. **Database** stores the optimized URL and public ID.
6. On update/delete, old images are removed from Cloudinary.

Benefits: faster loading, reduced storage, global CDN delivery.

10. API DEVELOPMENT

RESTful APIs were developed for all modules. All endpoints follow consistent patterns:

- **Base URL:** https://api.raritone.com or http://localhost:5000
- **Authentication:** Authorization: Bearer <accessToken>
- **Request/Response:** JSON format

Key API Endpoints (Examples)

Module	Method	Endpoint	Description
Auth	POST	/api/auth/signup	Register user
Auth	POST	/api/auth/login	Login → tokens
Product	GET	/api/products/search?keyword=shirt	Search products
Cart	POST	/api/cart	Add to cart
Order	POST	/api/orders	Create order
Wishlist	POST	/api/wishlist/add	Add to wishlist
Measurements	POST	/api/measurements	Save body measurements
Try-On	POST	/api/tryOnRoutes/upload/tryon	Upload try-on images

Full API documentation is available in the technical documentation.

11. TESTING

The system was tested using **Postman** and **MongoDB Compass**.

Test Coverage

Area	Tests Performed	Result
Authentication	Signup, login, token validation, password reset, Google OAuth	✓ Pass
Product CRUD	Create, read, update, delete; image upload; search/filter	✓ Pass
Cart	Add, remove, get cart; quantity accumulation	✓ Pass
Order	Create, retrieve, update status, delete	✓ Pass
Profile	Get, update bio, preferences, image upload/delete	✓ Pass
Wardrobe	Add, get, delete item (with image cleanup)	✓ Pass
Wishlist	Add, remove, get wishlist	✓ Pass (auth to be added)
Measurements	Save, update, get, delete	✓ Pass
Try-On	Upload two images, history, delete	✓ Pass
Security	JWT expiry, role access, rate limiting (100/15min)	✓ Pass

Known Issue

Wishlist routes currently lack authMiddleware – this will be fixed in the next iteration.

12. AI INTEGRATION PLANNING

The project was designed to support future AI services with minimal changes.

Planned AI Features

- **AI Avatar Generation** – Convert user uploads into 3D avatars (supports multiple avatars per user).

- **Personalized Recommendations** – Using ProductMapping (supported body types, compatibility) and user measurements.
- **Virtual Try-On AI** – Replace mock endpoints with real inference (VITON-HD, HR-VITON, or commercial APIs).
- **Real-Time Updates** – [Socket.IO](#) already implemented for try-on status events.

AI-Ready Data Structures

- User.avatarImage, bodyScanImage – for avatar generation.
 - Measurement – chest, waist, hip, etc. for size matching.
 - ProductMapping – supportedBodyTypes, avatarCompatibility.
 - TryOn – stores original and generated images.
-

13. SCALABILITY RESEARCH

Several scalability solutions were researched to handle future growth:

Technology	Purpose
Redis	Caching product catalog, user sessions, API responses
BullMQ / RabbitMQ	Queue for async tasks (avatar generation, try-on processing, emails)
Load Balancer (Nginx / AWS ELB)	Distribute traffic across multiple server instances
Docker	Containerization for consistent deployment
Kubernetes	Orchestration for auto-scaling and self-healing
MongoDB Atlas Scaling	Sharding by userId/productId, auto-backup

These technologies can be integrated without rewriting the existing codebase.

14. CHALLENGES FACED

Challenge	Solution
Designing a scalable architecture	Implemented modular, layered architecture with clear separation of concerns
Managing relationships between multiple modules	Used MongoDB references and Mongoose populate
Security implementation (JWT, roles, rate limiting)	Added authMiddleware, roleMiddleware, rate limiter, Helmet
Planning future AI workflows	Created AI-ready schemas (ProductMapping, TryOn, avatar fields) and Socket.IO placeholders
Image upload performance	Integrated Sharp for compression and Cloudinary for CDN delivery

15. KEY LEARNINGS

Throughout the internship, significant knowledge was gained in:

- **Node.js & Express.js** – Building REST APIs, middleware, error handling
- **MongoDB & Mongoose** – Schema design, indexing, population, relationships
- **JWT Authentication** – Access/refresh tokens, secure storage, role-based access
- **Google OAuth** – Social login integration
- **REST API Design** – Standardized endpoints, status codes, response formats
- **Backend Architecture** – Layered design, modularity, scalability
- **Security Best Practices** – Helmet, rate limiting, input validation (Joi), bcrypt
- **Cloud Storage** – Multer, Sharp, Cloudinary integration for image pipelines
- **Real-Time Communication** – [Socket.IO](#) basics
- **Testing** – Postman collections, edge-case testing
- **Documentation** – Writing technical docs for developers
- **Git & GitHub** – Branching, pull requests, collaboration
- **Team-Based Development** – Coordinating modules with other developers

16. FUTURE ENHANCEMENTS

Planned improvements include:

- **Full AI Avatar Generation** – Integrate with generative AI models.
- **Recommendation Engine** – Use ProductMapping + user measurements to suggest products.
- **Redis Caching** – Reduce database load for product catalog.
- **Queue Processing** – Offload try-on and avatar generation to background workers.
- **Docker Deployment** – Containerize the application for consistency.
- **AWS Infrastructure** – Deploy on EC2, S3, CloudFront for global scale.
- **Complete Wishlist Auth** – Add missing authMiddleware to wishlist routes.
- **Implement Avatar Controller** – Complete the avatar module with full CRUD.

17. TEAM CONTRIBUTIONS

The Raritone Backend Project was developed collaboratively by a team of backend developers. Each team member contributed to specific modules while also participating in testing, documentation, integration, and overall system development.

Nandini Bhimineni (Backend Developer & Integration Coordinator)

- Backend integration, cart, order, measurement, avatar, product mapping modules
- Database relationship design, API testing, cloud research, scalability planning, AI integration planning

Vagdevi Malineni

- Authentication module, JWT security, RBAC, profile module, wardrobe module, protected routes

Bharath Kumar (Me)

- Authentication module support, wishlist module development, API validation implementation, security enhancements, avatar module support, backend testing support

Vishnu Vardhan Reddy

- Wishlist module development, virtual try-on support, authentication support, API testing

Meka Hrishith Teja Chowdary

- Product module development, order module, virtual try-on support, product management

Nainisha Bandari

- Profile module, wardrobe module, user preference management, profile API development

Collective Team Contributions

- REST API development, MongoDB design, JWT + Google OAuth integration, Cloudinary + image upload, error handling middleware, input validation, security features, documentation, GitHub collaboration, Postman testing, architecture design, AI & scalability research.
-

18. MY CONTRIBUTIONS (BHARATH KUMAR)

Overview

As a backend developer in the Raritone project, I focused on **authentication support**, **wishlist module development**, **API validation**, **security enhancements**, **avatar module support**, and **backend testing**.

Wishlist Module (Lead)

- Designed wishlist schema (userId + products array).
- Implemented addToWishlist, removeFromWishlist, getWishlist controllers.
- Ensured duplicate prevention and populated product details.
- Created RESTful routes (/api/wishlist/add, /remove/:productId, /).

Authentication Module (Support)

- Assisted in signup/login logic, JWT generation, and Google OAuth integration.
- Helped implement password reset with Nodemailer.

API Validation Implementation

- Created Joi validation schemas for auth and product endpoints.
- Integrated validation middleware into routes to reject malformed requests.

Security Enhancements

- Configured global rate limiting (100 requests per 15 minutes).
- Enabled Helmet for secure HTTP headers.
- Verified role-based access control middleware.

Avatar Module Support

- Reviewed avatar schema design and provided input on routes.

Backend Testing Support

- Tested all authentication, wishlist, and validation endpoints using Postman.
- Verified error handling, edge cases, and rate limiting.
- Identified missing auth on wishlist routes (will be fixed).

Key Achievements

- **Complete wishlist module** – fully functional and documented.
 - **Validation layer** – reduced invalid requests and improved data integrity.
 - **Security hardening** – rate limiting and Helmet integrated.
 - **Testing** – ensured reliability of core features.
-

19. CONCLUSION

The Raritone Backend Project successfully established a **secure, scalable, and AI-ready backend foundation** for a virtual fashion platform. The project implemented essential e-commerce and personalization features – authentication, product management, shopping cart, orders, wishlist, wardrobe, measurements, try-on, and image handling – while preparing the system for future AI-driven enhancements.

Through this internship, I gained valuable hands-on experience in **Node.js, Express, MongoDB, JWT, Google OAuth, Cloudinary, security practices, testing, and team collaboration**. The project demonstrated how to build a production-ready backend with modular architecture, scalability research, and AI integration planning.

I am grateful to the Raritone team for this opportunity, which has strengthened my technical skills and prepared me for real-world software development challenges.